

Sonora

Estado da migração do backend: Base44 → Supabase

Este documento resume o trabalho de migração do backend do Sonora, que deixou de depender da plataforma Base44 (entretanto descontinuada) e passou a correr inteiramente sobre Supabase (base de dados Postgres, autenticação, storage e realtime) mais duas Supabase Edge Functions para as duas únicas operações que precisam de uma chave secreta (envio de email e geração de texto por IA). O Vercel passa a servir apenas o build estático do Vite — sem funções serverless, sem SSR — o que mantém o consumo de recursos ao mínimo possível.

1. O que já está implementado

1.1 Base de dados (supabase/schema.sql)

- Tabelas para todas as entidades do catálogo Base44: **songs, albums, artists, podcasts, podcast_episodes**.
- Tabelas pessoais: **favorites, listening_history**.
- Tabelas sociais: **friendships, friend_activity, profiles**.
- **playlists** (públicas/privadas) e **user_settings** (equivalente aos campos custom do User do Base44: role, bio, cover_photo_url, display_name, photo_url, share_activity, access_verified).
- Row-Level Security em todas as tabelas, replicando as regras do Base44 (dono só vê o seu; catálogo de leitura aberta; playlists públicas/privadas; friend_activity com leitura aberta e escrita só do dono).
- Trigger que cria automaticamente a linha de **user_settings** quando um novo utilizador se regista.
- Publicação realtime (playlists, friend_activity, friendships) — mantém a sidebar e a página Amigos sincronizadas sem reload, tal como no Base44.
- Bucket de storage **sonora-uploads** (capas de playlist, foto/capa de perfil), com política de leitura pública e escrita restrita à pasta do próprio utilizador.

1.2 Camada de compatibilidade (src/api/)

- **base44Client.js** foi reescrito para expor exatamente a mesma forma que a app já usava (`base44.auth.*`, `base44.entities.<Entidade>.*`, `base44.integrations.Core.*`), agora ligado ao Supabase por baixo.
- Resultado prático: as ~25 páginas/componentes que já chamavam essas funções continuam a funcionar sem alterações — só a implementação por trás mudou.
- **entities.js**: `list / filter / get / create / update / delete / deleteMany / bulkCreate / subscribe`, com tradução do pequeno subconjunto de operadores Mongo-like usados no código (igualdade, `$in`, `$gte`, etc.) para queries PostgREST.
- **auth.js**: login por email/password, registo com código OTP de 6 dígitos, login social (Google/GitHub/Meta), pedido e reposição de palavra-passe, `me()/updateMe()`.
- **integrations.js**: `UploadFile` (Supabase Storage), `SendEmail` e `InvokeLLM` (via Edge Functions); `GenerateImage/GenerateVideo/GenerateSpeech/TranscribeAudio/ExtractDataFromUploadedFile` ficam como stubs "não implementado" — nada no código atual os chama.

1.3 Edge Functions (supabase/functions/)

- **send-email**: envia convites de amizade e reportes de problema via Resend. Verifica sempre que quem chama está autenticado antes de enviar.
- **invoke-llm**: substitui o antigo pedido de "letra completa" da música (que reproduziria letras com direitos de autor). Gera agora um texto curto e original sobre o tema/ambiente da música, deixando claro que não é a letra — o painel de letras no player foi ajustado em conformidade.

- **fetch-podcast-feed** (novo): vai buscar e faz parse ao feed RSS de um podcast no servidor (a maioria dos feeds bloqueia fetch direto do browser por CORS), devolvendo os episódios já limpos — título, URL direta do .mp3, descrição, data e duração. Não precisa de nenhuma chave secreta. Tem proteções básicas contra SSRF (só aceita http/https, bloqueia hosts internos/privados) e limites de tamanho/tempo no pedido.

1.4 Podcasts — episódios reais via RSS

- Corrigido um problema em que subscrever um podcast (via pesquisa iTunes) nunca ia buscar os episódios — a lista ficava sempre vazia. Agora, ao adicionar um podcast, o feedUrl devolvido pela iTunes Search API é enviado à Edge Function **fetch-podcast-feed**, e os episódios voltam já prontos a gravar em **podcast_episodes** (bulkCreate).
- Limite de 300 episódios mais recentes por podcast (a lista na interface já só mostra os 100 mais recentes; isto dá margem sem sobrecarregar feeds muito grandes).
- Descrição dos episódios é limpa de HTML antes de gravar, para não aparecerem tags soltas na interface.
- Rádios (Radio.jsx) — já correspondiam à abordagem documentada (Radio Browser API filtrada por countryCode=PT); não foram necessárias alterações.
- Entrevistas (InterviewsSection.jsx, dentro da página Podcasts) — já funcionavam de forma independente via iTunes Search API, sem dependência de backend; não foram necessárias alterações.

1.5 Autenticação e páginas

- **AuthContext.jsx** reescrito para sessões Supabase (o fluxo antigo dependia de endpoints da plataforma Base44 que deixaram de existir).
- **ResetPassword.jsx** reescrito para o fluxo de recuperação de password do Supabase (sessão de recovery em vez de token na query string).
- Login/Registo (email+password, OTP, Google/GitHub/Meta) não precisaram de alterações — já usam exatamente as chamadas que o novo base44 .auth implementa.

1.6 Algoritmos (Smart Shuffle, DJ Mode, Recomendações)

- **Smart Shuffle** (recommendations.js): shuffle anti-clustering — em vez de aleatório puro, evita tocar duas músicas seguidas do mesmo artista sempre que matematicamente possível. Validado fora da app com 300 simulações: numa lista muito desequilibrada atinge exatamente o mínimo teoricamente possível de repetições consecutivas (29.41%, contra ~45% de um shuffle aleatório puro na mesma lista); em listas equilibradas, 0%. Já ligado ao shuffle da fila de reprodução e ao shuffle de playlists.
- **Modo DJ** (PlayerContext.jsx): curva de crossfade corrigida de linear para equal-power (seno/cosseno), a curva tecnicamente correta em DSP de áudio para a transição não soar mais baixo a meio. Mantido o motor de um único elemento de áudio (em vez de dois decks em simultâneo) — essa reescrita maior fica para quando houver catálogo real para testar, evitando arriscar a reprodução atual sem forma de validar ao vivo.
- **Recomendações**: getRecommendations (rádio/estação) e a nova getPlaylistSuggestions ("Músicas sugeridas", agora baseada no centróide de toda a playlist, não só na primeira música) já sabem usar BPM/energia/valência quando existirem — com fallback automático para artista/género quando não existirem.
- Colunas opcionais bpm, energy, valence, key_signature preparadas em **supabase/schema_audio_features.sql** para quando esses dados existirem.

1.7 Limpeza

- Removida a página "Barra Secret" (exportação de código/documentação) e tudo o que só ela usava: docsContent.js, pdfBuilder.js, zipBuilder.js, a rota /secret-bar e o gatilho de pesquisa "secret".
- Removido **OAuthConsent.jsx** e **app-params.js** — código morto específico do fluxo de consentimento MCP do Base44, sem equivalente no Supabase e sem rota nenhuma a apontar para lá.

- Removida a dependência `jspdf` (só usada pelo código removido) e as dependências `@base44/sdk` / `@base44/vite-plugin`, substituídas por `@supabase/supabase-js`.
- Criado `.gitignore` (não existia — `node_modules/`, `dist/` e `.env` já não vão para o Git).
- Build (`npm run build`) e lint verificados sem erros novos.

2. Pendente do teu lado

Checklist de configuração na Dashboard do Supabase / Vercel. Nada disto precisa de código novo — são passos de configuração.

Item	Estado	Nota
Correr supabase/schema.sql no SQL Editor	Feito	Confirmado por ti.
.env local (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY)	Feito	Confirmado — o ficheiro .env já existe e a app liga ao Supabase.
Mesmas variáveis no projeto Vercel (Production + Preview)	Pendente	Necessário antes do primeiro deploy em produção.
Deploy da Edge Function fetch-podcast-feed	Pendente — bloqueia Podcasts	supabase functions deploy fetch-podcast-feed · sem secrets necessários. Sem isto, inscrever um podcast grava o podcast mas fica sem episódios.
Confirmar manualmente a conta de teste (SQL abaixo)	Pendente	Necessário para testar páginas autenticadas (Podcasts/Rádio) sem depender do email de OTP.
Template de email "Confirm signup" com {{ .Token }}	Adiado	Combinado: fica com o padrão do Supabase por agora. Nota: o padrão do Supabase não inclui {{ .Token }}, por isso o registo por email fica sem código OTP até este passo ser feito — não tem ligação nenhuma com o Resend, é uma definição própria do Auth do Supabase.
Deploy das Edge Functions send-email e invoke-llm	Adiado	supabase functions deploy send-email invoke-llm — combinado para quando ligares o Resend.
Secrets RESEND_API_KEY e ANTHROPIC_API_KEY	Adiado	Só bloqueia convites de amigos, reportes e o painel de letras — não bloqueia o resto da app.
OAuth providers (Google / GitHub / Facebook)	Adiado	Combinado: trata disto mais tarde. O código dos botões já está pronto e à espera das credenciais.

Comando para confirmar manualmente uma conta (por email)

Corre no SQL Editor do Supabase, substituindo o email. Marca a conta como confirmada sem precisar do código enviado por email — útil para contas de teste enquanto o template de OTP não está configurado.

```
update auth.users
set email_confirmed_at = now()
where email = 'exemplo@dominio.com';
```

3. Verificação — tudo o que não depende de música/letras

Fiz uma revisão de código completa a todas as páginas e componentes que não dependem do catálogo de músicas ou de letras: Home, Biblioteca (recentes/artistas/álbuns/favoritos/playlists), Pesquisa, Estatísticas, Amigos (convites, perfil de amigo, atividade), Podcasts, Rádio, Perfil, Playlist (criar/editar/reordenar/partilhar), PlayerBar, Now Playing, e o layout/sidebar com realtime. Confirmação feita a dois níveis:

- **Estático** (o que consigo garantir sem sessão ao vivo): todas estas páginas tratam corretamente o caso de dados vazios (catálogo ainda sem músicas) — mostram estado vazio/skeleton em vez de rebentar; todas as chamadas a base44.* usadas correspondem exatamente ao que a camada de compatibilidade Supabase implementa; npm run build e npm run lint continuam sem erros novos.

- **Ao vivo** (login → navegar → interagir): fica por confirmar. Tentei duas vezes com a conta de teste (magicard.mail+sonoratest@gmail.com) e continua "Email not confirmed" — o comando SQL da secção 2 ainda não foi corrido. Tentei também usar a Supabase CLI, já autenticada nesta máquina, para confirmar a conta diretamente, mas os projetos ligados a essa conta chamam-se "**NexusCraft**" e "**FocusGrid**" — nenhum "Sonora". Não corri nada contra esses projetos por não saber se algum deles é o teu backend do Sonora (ou se está noutra conta). Preciso que confirmes qual é, ou corrás tu o comando SQL, para fechar este último passo de verificação.

4. Funcionalidades por implementar / decisões futuras

- **Catálogo** (songs, albums, artists) — tabelas e RLS prontas, mas vazias. Combinado: definimos a forma de povoar isto mais tarde.
- **Core.GenerateImage / GenerateVideo / GenerateSpeech / TranscribeAudio / ExtractDataFromUploadedFile** — ficam por implementar (stubs) até decidirmos se/quando são precisos.
- **Modo DJ avançado (crossfade dual-deck), Karaoke com isolamento vocal e letras sincronizadas via LRCLIB** — descritos no PDF "Engine Streaming — Arquitetura Completa" que partilhaste; o Modo DJ atual (fade simples no fim/início da faixa) já existe em PlayerContext.jsx, mas a versão dual-deck com beatmatching, o karaoke por DSP/IA e a sincronização linha-a-linha via LRCLIB não foram tocados nesta ronda — ficam para quando quiseses avançar com isso especificamente.
- **Fonte de áudio para Músicas & Álbuns** — esse mesmo PDF propõe resolver o áudio das músicas via scraping do YouTube (yt-dlp/Piped). Não vou implementar essa parte: extrair e servir catálogo comercial sem autorização viola os termos do YouTube e os direitos das editoras. Quando chegarmos ao catálogo, precisamos de uma fonte de áudio com licença (upload próprio, ou um serviço com API licenciada).

Gerado automaticamente a partir do trabalho realizado nesta sessão. Ficheiros de referência: supabase/schema.sql, .env.example, src/api/*, supabase/functions/*.